WEEK 2 · DAY 1, PART 1 · OVERVIEW

# NumPy, Part 1




Created by **Jeff Lenamon**

ABOUT 50 MIN WITH EXERCISES / 4 SECTIONS / 5 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

# The Desk Lead's Eighth Note

Last week every column question took a loop. This week, one line each, on the four bonds of Days 1 to 4, so every answer checks against last week's: each position's cost and income, each bond's current yield, the top yielder, and the average coupon that matches our cash. Then make me a year of test yield moves, and tell me what QVNE returned this week.

ABOUT 50 MIN WITH EXERCISES / 4 SECTIONS / 5 EXERCISES

# Your Route Through Day 1, Part 1

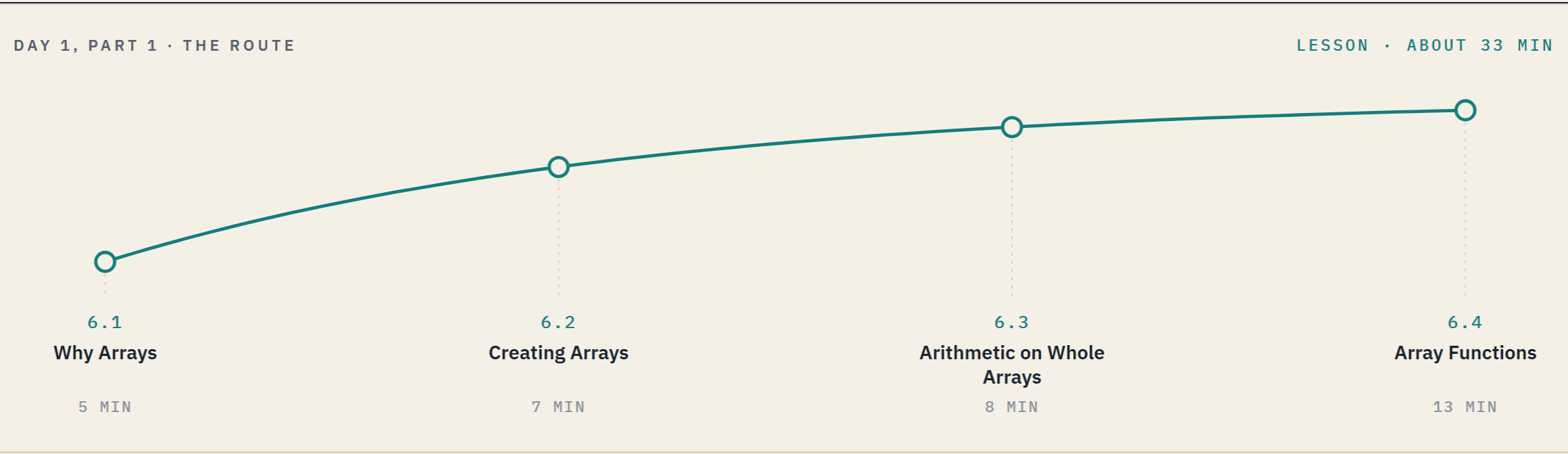



FIGURE 1 Four sections, in order, with about how long each takes.

**Time. About 33 minutes for the lesson, then about 15 for the five exercises.**

# 6.1 Why Arrays

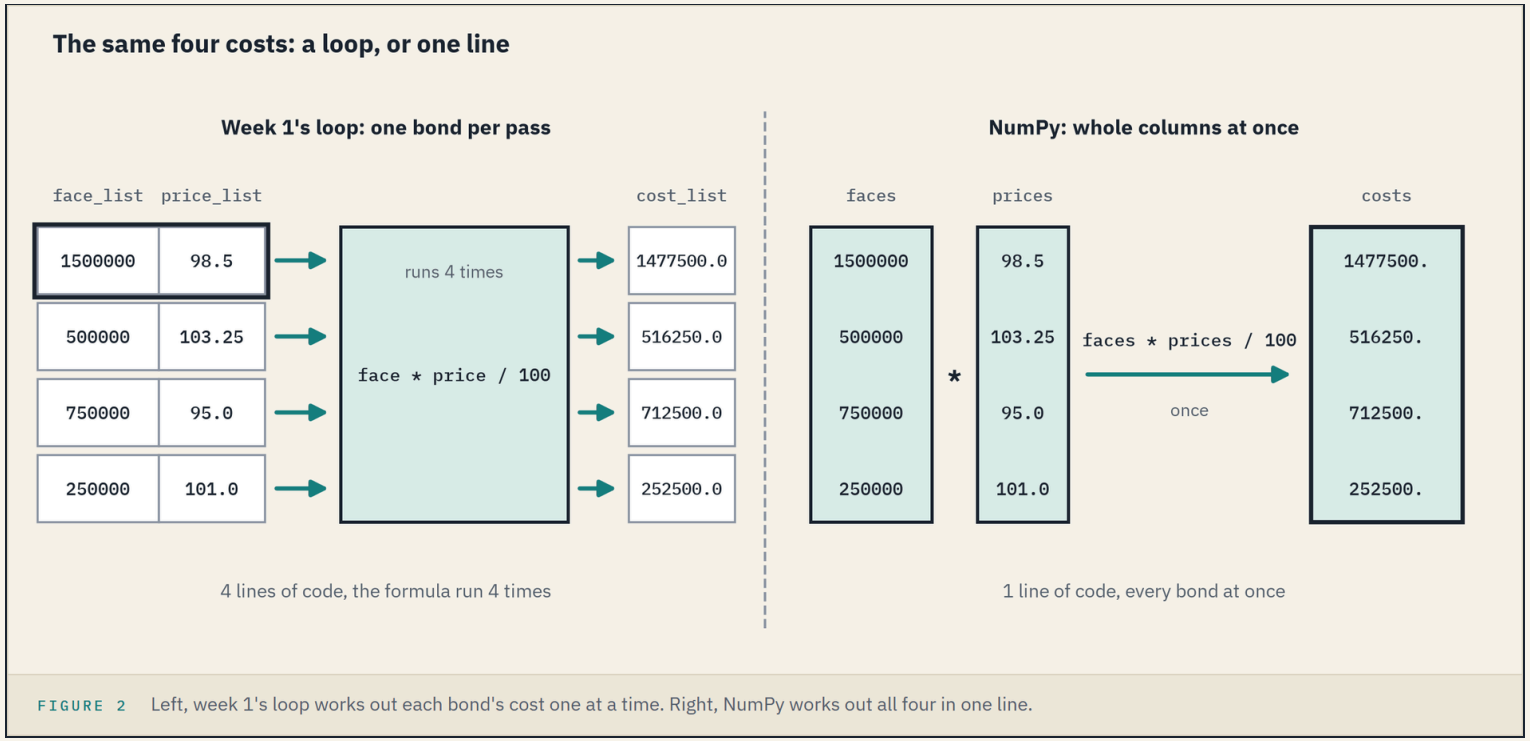



FIGURE 2 Left, week 1's loop works out each bond's cost one at a time. Right, NumPy works out all four in one line.

ONE LINE FOR THE BOOK

```
faces = np.array(face_list)
prices = np.array(price_list)
costs = faces * prices / 100
print(costs)
```

```
[1477500.  516250.  712500.  252500.]
```

EXCEL SIDE BY SIDE

With face in B and price in C, `=B2*C2/100` filled down gives the four costs.

**With arrays, * multiplies, value by value.**

# 6.2 Creating Arrays

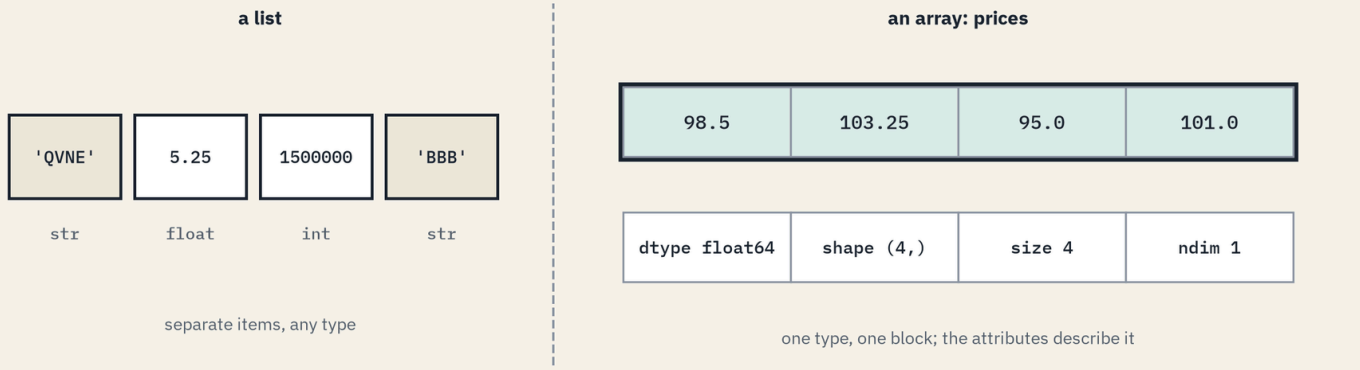



FIGURE 3 A list holds separate items of mixed types; an array is one block of float64, with its shape, size, and ndim.

### LOOK INSIDE AN ARRAY

```
print('coupons:', coupons.dtype)
print('faces:', faces.dtype)
print('tickers:', tickers.dtype)
```

```
coupons: float64
faces: int64
tickers: <U4
```

### AN ARRAY FROM A LIST

```
print(type(prices))
print(tickers)
```

```
<class 'numpy.ndarray'>
['QVNE' 'DNMR' 'PLWK' 'HLVF']
```

**It looks like a list. Look inside.**

# Runs of Numbers

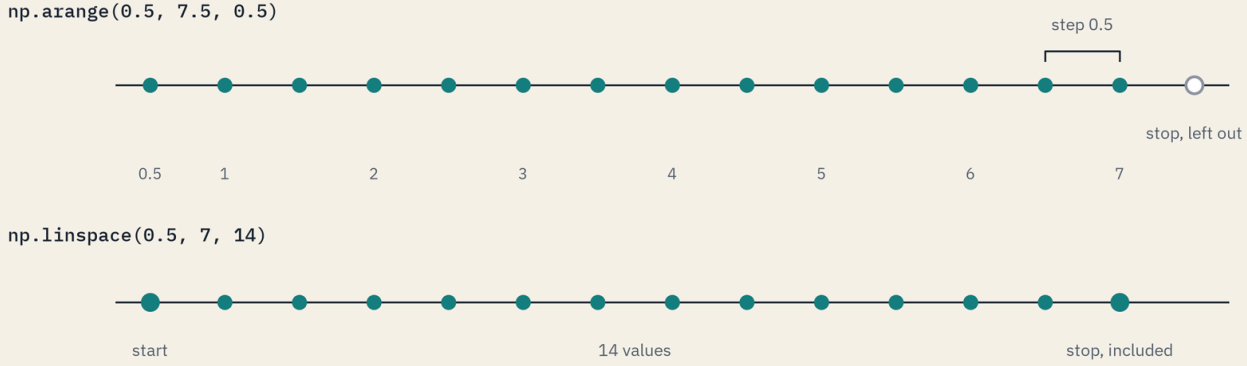



FIGURE 4 The same 14 payment times two ways: arange by step, its stop left out; linspace by count, both ends included.

STEP KNOWN

```
print(np.arange(95, 106, 1))
```

```
[ 95  96  97  98  99 100 101 102 103 104 105]
```

EXCEL SIDE BY SIDE

In Microsoft 365 and Excel 2021, `=SEQUENCE(11, 1, 95, 1)` spills the 11 prices: 11 rows, 1 column, from 95, a step of 1.

**Step known:** `np.arange()`. **Count known:** `np.linspace()`.

# 6.3 Arithmetic on Whole Arrays

### INCOME AND YIELD

```
incomes = faces * coupons / 100
print(incomes)
current_yields = coupons / prices * 100
print(current_yields)
```

```
[78750. 30625. 35625. 18750.]
[5.32994924 5.93220339 5.         7.42574257]
```

### THE TOTALS

```
print('Total cost:', costs.sum())
print('Annual income:', incomes.sum())
```

```
Total cost: 2958750.0
Annual income: 163750.0
```

### EXCEL SIDE BY SIDE

| QUESTION | PYTHON | EXCEL |
|---|---|---|
| Cost of every position | `faces * prices / 100` | `=B2*C2/100` filled down |
| Current yield of every bond | `coupons / prices * 100` | `=D2/C2*100` filled down |
| Total cost | `costs.sum()` | `=SUM(E2:E5)` |
| Total cost in one step | `(faces * prices / 100).sum()` | `=SUMPRODUCT(B2:B5, C2:C5)/100` |

**2,958,750 and 163,750: Day 4's tied totals, with no loop.**

# Arrays That Do Not Line Up

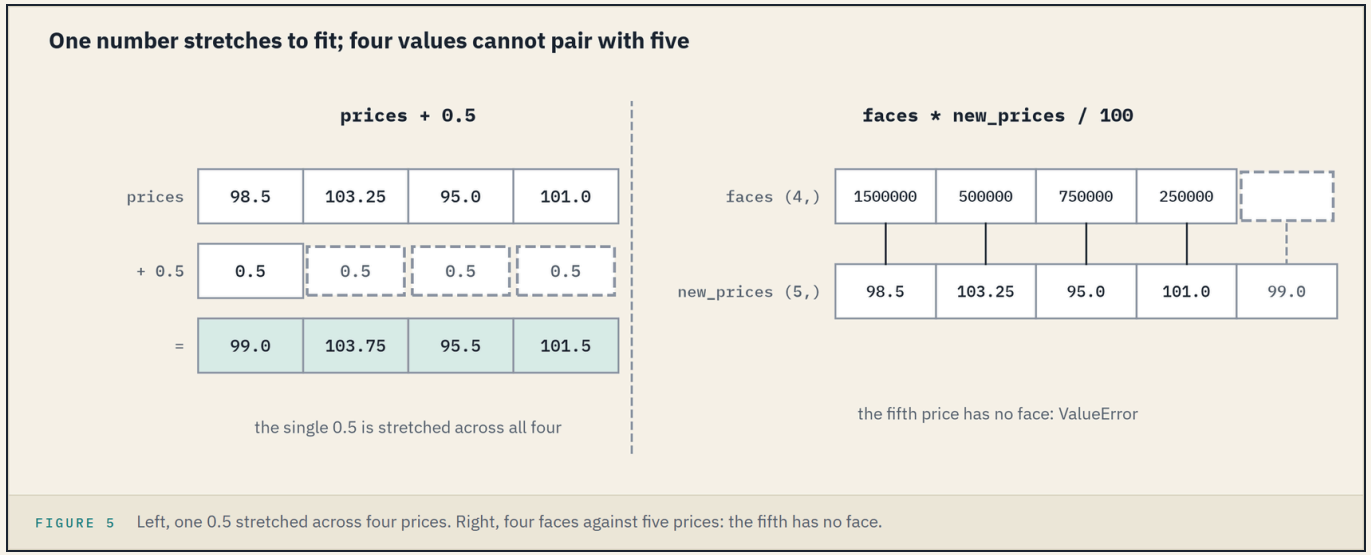



FIGURE 5 Left, one 0.5 stretched across four prices. Right, four faces against five prices: the fifth has no face.

### THE SHAPES DIFFER

```
new_prices = np.array([98.5, 103.25, 95.0, 101.0, 99.0])
faces * new_prices / 100
```

### A ZERO PRICE

```
entered_prices = np.array([98.5, 0.0, 95.0, 101.0])
yields_with_zero = coupons / entered_prices * 100
print(yields_with_zero)
```

```
[5.32994924        inf 5.          7.42574257]
```

**NumPy stops when the shapes do not match.**

# 6.4 Array Functions

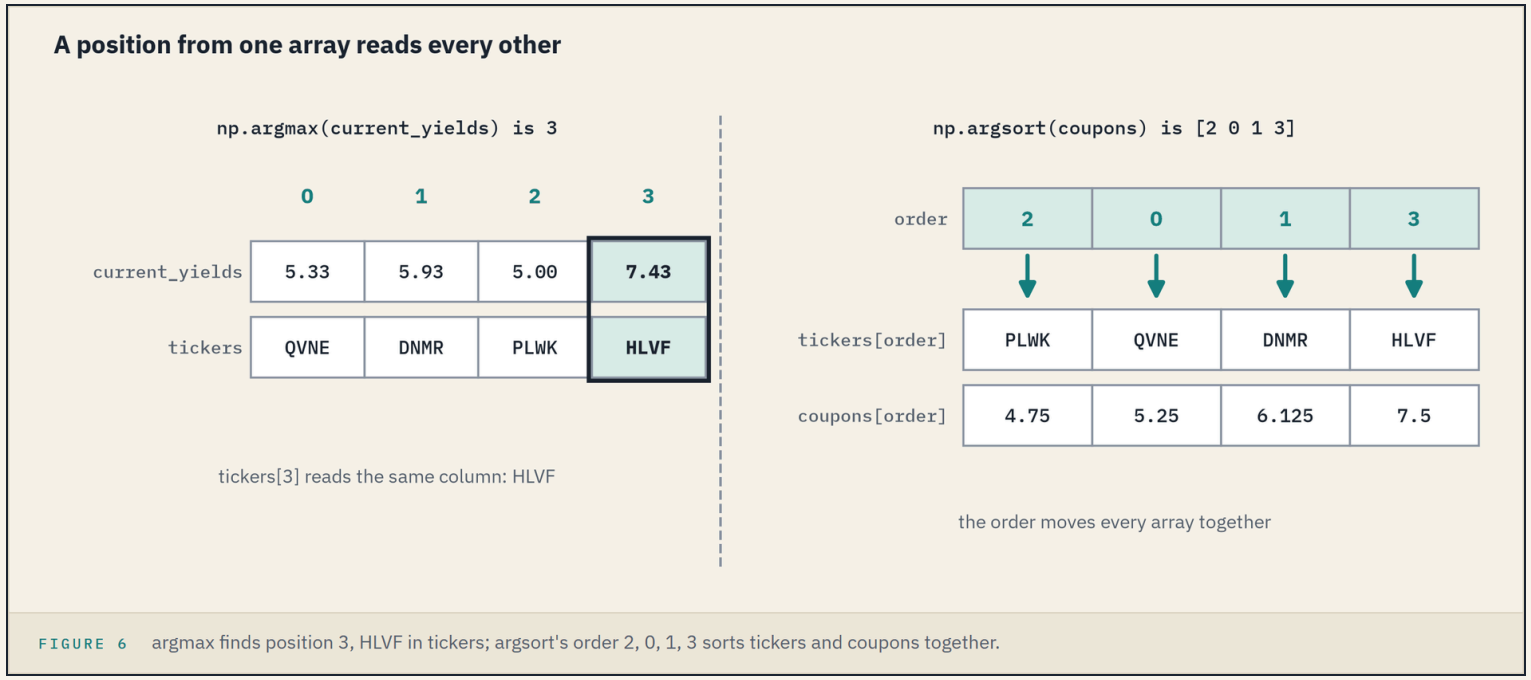



FIGURE 6 argmax finds position 3, HLVF in tickers; argsort's order 2, 0, 1, 3 sorts tickers and coupons together.

### FIND THE BOND

```
top = np.argmax(current_yields)
print(top, tickers[top])
```

```
3 HLVF
```

### SORT WITH THE POSITIONS

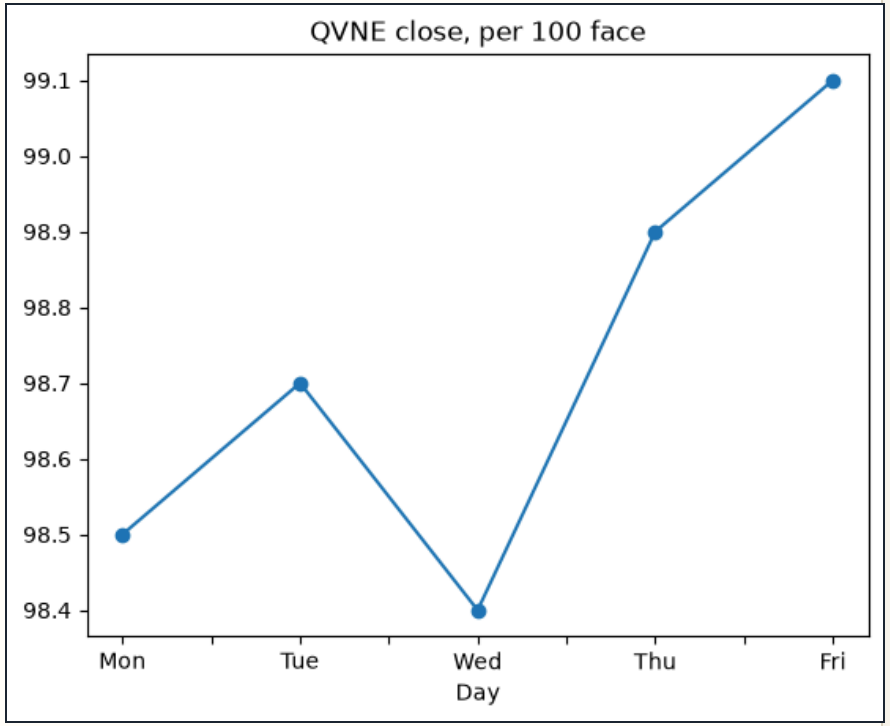
```
order = np.argsort(coupons)
print(order)
print(tickers[order])
```

```
[4.75  5.25  6.125 7.5  ]
[2 0 1 3]
```

**Positions keep each bond together. Now the average the note asked for.**

# A Week of Returns

## QVNE'S WEEK




## CHANGES AND RETURNS

```
price_changes = np.diff(qvne_week)
print(np.round(price_changes, 2))
simple_returns = price_changes / qvne_week[:-1]
print(np.round(simple_returns * 100, 4))
```

```
[ 0.2 -0.3  0.5  0.2]
[ 0.203  -0.304   0.5081  0.2022]
```

## LOG RETURNS

```
log_returns = np.log(qvne_week[1:] / qvne_week[:-1])
print(np.round(log_returns * 100, 4))
```

```
[ 0.2028 -0.3044  0.5068  0.202 ]
```

**Log returns add up across days; simple returns do not.**

# You Can Now

- Work out cost, income, and current yield for every bond in one line.
- Say why `*` repeats a list and multiplies an array.
- Build arrays with `np.arange()`, `np.linspace()`, `np.zeros()`, and `np.ones()`.
- Summarize a book: totals, the top bond by name, a face-weighted average.
- Compare decimals with `np.isclose()`, never `==`.
- Read a divide-by-zero warning, and see why one `inf` spoils a total.
- Use a standard deviation, seeded random numbers, and simple and log returns.

NEXT
WEEK 2 · DAY 1, PART 2

## NumPy, Part 2

Picking out bonds by position and by condition, holding a table in one array, and pricing a bond from its cash flows.